

CSC270 · DATABASE SYSTEMS · LECTURE 32

Course Review · Capstone

Course Review

A tour of all 8 units, following one running example — a university's Student/Course enrollment data — through every stage a real system takes it.

Agenda: A Tour of All 8 Units

- 1 One running example, threaded through all 8 units
- 2 Units 1–3: Foundations, the Relational Model, Algebra & Calculus
- 3 Units 4–5: ER/EER Modeling and Normalization
- 4 Unit 6: Views, Security, and Indexing
- 5 Unit 7: NoSQL and MongoDB
- 6 Unit 8: Transaction Management
- 7 The whole course as one thread, and where it leads next

The Running Example

A university needs to record which students are enrolled in which courses. It's small enough to hold in your head in full, and rich enough that every unit of this course had something genuine to say about it.

Thirty-one lectures ago, this course opened with a simple question: what actually is a database, and why isn't "just a bunch of files" good enough? This lecture walks back across all eight units to answer it in full.

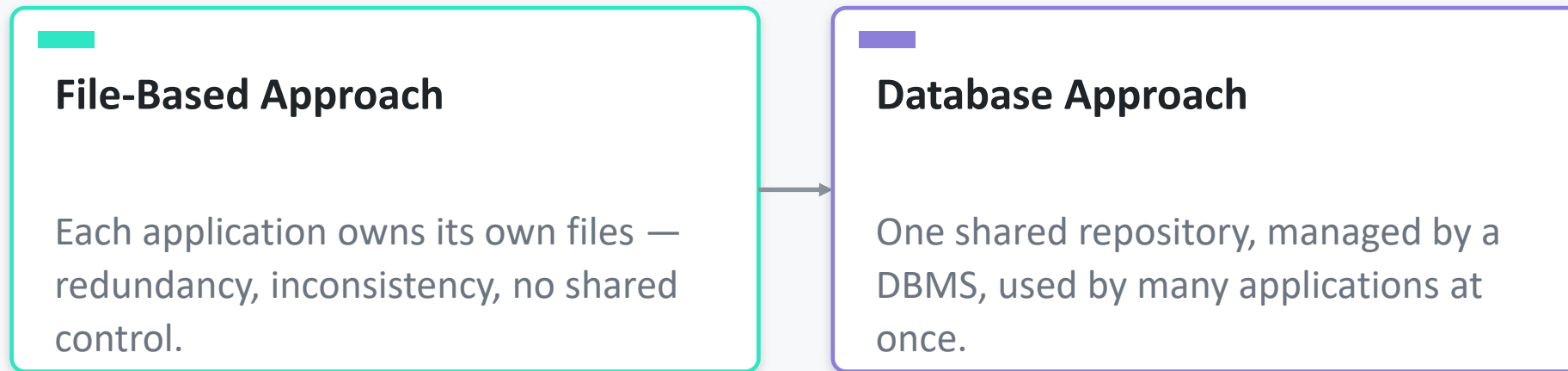
STUDENT — ENROLLS (M:N) — COURSE

- `studentId, name, major`
- `courseId, title, seatsAvailable`
- `Enrollment(studentId, courseId, grade)`

"Which CS students are enrolled in a Database Systems course?"

Unit 1 — Foundations of Database Systems

A file-based approach (a plain spreadsheet of student records, another of course records, no coordination) suffers from redundancy, inconsistency, and poor access control. The database approach fixes this with one layer of software between every application and the raw data.



Unit 2 — The Relational Model

Data lives in relations (tables) — sets of tuples (rows) over a fixed set of attributes (columns). Integrity constraints keep a relation trustworthy: domain constraints, entity integrity (no NULL primary key), and referential integrity.

Student (studentId, name, major)

<u>studentId</u>	name	major
S001	Amina Raza	Computer Science
S002	Bilal Hussain	Computer Science

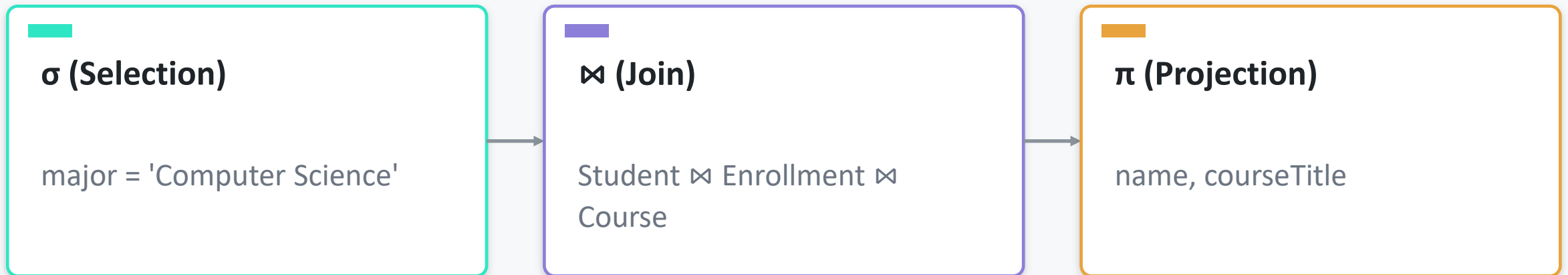
Underlined = primary key

Domain, entity, and referential integrity all apply to this one relation.

Unit 3 — Relational Algebra and Calculus

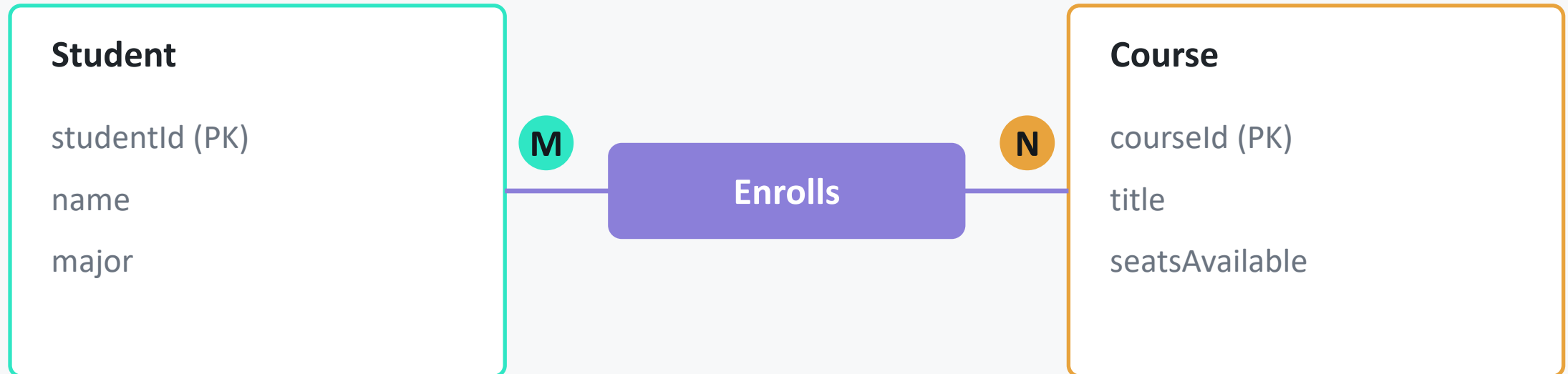
A formal query language built from a small set of operators — selection, projection, join, union, division — each taking whole relations as input and producing a relation as output. That's exactly what lets these operators compose.

“Which Computer Science students are enrolled in a Database Systems course?”



Unit 4 — Data Modeling: ER and EER

The running example first takes shape here, at the conceptual level: Student and Course as entities, connected by an Enrolls relationship with M:N cardinality — refined further with EER tools like specialization where needed.



Unit 5 — Normalization, Part 1: The Problem

A naive single table repeats the student's name and course's title on every row — the update-anomaly risk normalization exists to eliminate. Decomposing it into 1NF → 2NF → 3NF form removes the redundancy entirely.

Unnormalized — repeats name and title

studentId	name	courseId	title	grade
S001	Amina Raza	CS270	Database Systems	A
S001	Amina Raza	CS211	Algorithms	B+

Unit 5 — Normalization, Part 2: The Fix

3NF — Enrollment references, doesn't repeat

studentId	courseId	grade
S001	CS270	A
S001	CS211	B+

A fresh Enrollment table — holding only the foreign keys and any enrollment-specific attribute, like the grade — matches the ER diagram's M:N relationship as its own relation, exactly as Unit 4's modeling anticipated.

Unit 6 — Views & Security (Part 1 of 2)

Operationalizing the normalized schema for real applications.

A view exposes a student's transcript (Student ⋈ Enrollment ⋈ Course — the same join from Unit 3) as one queryable object; a GRANT limits who can read it.

VIEW StudentTranscript

Student ⋈ Enrollment ⋈ Course, exposed as one queryable object.



GRANT SELECT ... TO registrar_staff

Restricts who can read the view.

Unit 6 — Indexing (Part 2 of 2)

Operationalizing the normalized schema for real applications.

An index on Enrollment.studentId turns “find this student's enrollments” from a full scan into a direct lookup.

INDEX ON Enrollment(studentId)

Makes “find this student's enrollments” fast.

Unit 7 — NoSQL and MongoDB

Stepping outside the relational model. A document database like MongoDB would happily store a student's enrollments denormalized, embedded directly inside the student's own document — the opposite instinct from Unit 5 — because MongoDB's strength is reading one student's entire profile in a single lookup, without a join.

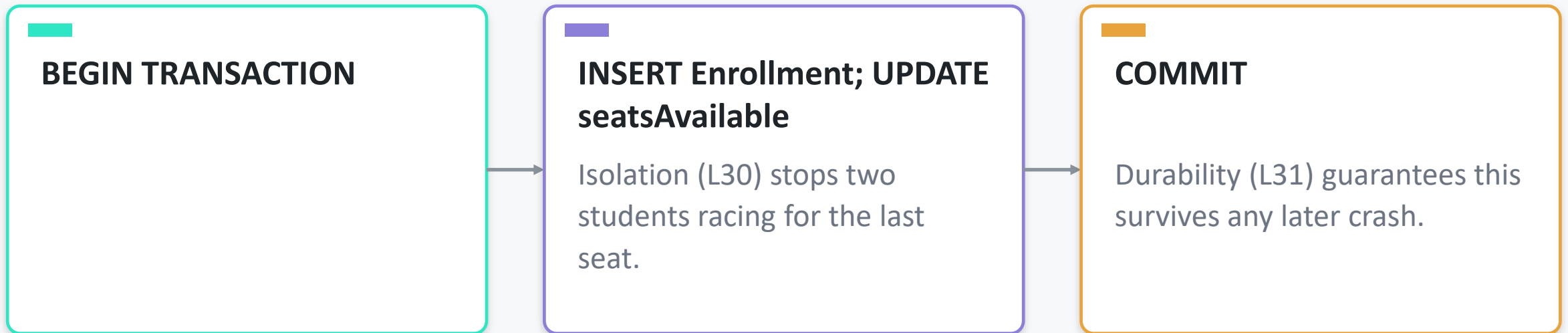
Neither choice is universally “correct” — each is the right tool for a different access pattern.

students COLLECTION — ONE EMBEDDED DOCUMENT

```
{
  "_id": "S001",
  "name": "Amina Raza",
  "major": "Computer Science",
  "enrollments": [
    { "courseId": "CS270",
      "title": "Database Systems",
      "grade": "A" },
    { "courseId": "CS211",
      "title": "Algorithms",
      "grade": "B+" }
  ]
}
```

Unit 8 — Transaction Management

Wrapping the whole thing in safety guarantees. Enrolling a student is, underneath, at least two writes — insert the Enrollment row and decrement Course.seatsAvailable — and both must succeed or neither should.



This is exactly Lecture 29's bank-transfer pattern, Lecture 30's concurrency control preventing a lost-seat race, and Lecture 31's recovery manager guaranteeing durability — applied to a single enrollment.

The Whole Course, as One Thread (Part 1 of 2)

Unit 1 — Foundations

A DBMS, not loose files, manages Student and Course data

Units 2–3 — Relational Model, Algebra & Calculus

Data lives in constrained relations; queries are composed operators over them

Unit 4 — ER/EER Modeling

Student —Enrolls(M:N)— Course, modeled conceptually first

The Whole Course, as One Thread (Part 2 of 2)

Unit 5 — Normalization

The M:N relationship becomes its own redundancy-free Enrollment table

Unit 6 — Views, Security, Indexing

A transcript view, access control, and a fast lookup path

Unit 7 — NoSQL Contrast

The same data, deliberately denormalized, for a different access pattern

Unit 8 — Transactions

Every write to this data, wrapped in ACID guarantees

Where This Leads Next: Distributed Databases

This course deliberately stopped at a single DBMS instance, serving requests one transaction at a time (however concurrently).

● Distributed Databases

What happens when data is spread across multiple machines, and a transaction might update rows on two at once — needing its own version of Atomicity and Isolation, across a network that can partially fail.

Where This Leads Next: Data Warehousing & OLAP

This course deliberately stopped at a single DBMS instance, serving requests one transaction at a time (however concurrently).

- **Data Warehousing & OLAP**

This course's relational model optimized for many small, fast transactions (OLTP); a data warehouse instead optimizes for huge, read-heavy analytical queries across a company's entire history.

Where This Leads Next: Query Optimization

This course deliberately stopped at a single DBMS instance, serving requests one transaction at a time (however concurrently).

- **Query Optimization**

Every SQL query this course wrote was silently run through a query optimizer choosing how to execute the relational algebra behind it — which index, which join order.

DATABASE SYSTEMS · CSC270

Course Complete

You started this semester with a question about files and spreadsheets, and built, unit by unit, a complete and rigorous answer: how to model data correctly, prove it's free of redundancy, query it with a composable algebra, secure and speed up access to it, consider a document-based alternative when it fits, and guarantee every change to it is safe — even against concurrent access and outright hardware failure.

That habit of mind — is this schema modeled correctly, normalized enough, indexed and secured, and every write actually safe — is what this course was built to give you.

Congratulations on completing Database Systems.